

AI TRAINING – DAY 6 ASSIGNMENT

Python Functions: Define → Pass → Return → Reuse

Learning Guidance + Refactoring Practice + Function-Based Utility Module

Level	Beginner → Medium
Suggested Time	3–4 hours
Core Topics	functions, parameters, return values, default arguments, *args, **kwargs
Daily Deliverable	Function-Based Utility Module
Main Focus	Turn repeated code into reusable functions

Important: What Is the Student Deliverable?

This assignment is your learning guidance. You do not need to submit a separate file for every activity. Your final Day 6 submission is ONE notebook containing your learning and testing, plus ONE Python utility module containing the final reusable functions.

```
Day_06_Function_Practice_YourName.ipynb
```

```
day_06_utility_module.py
```

Learning Goals

- Understand why functions are used in programming.
- Explain functions, parameters and return values in your own words.
- Create functions and call them correctly.
- Pass information using parameters.
- Return results using return.
- Use default arguments.
- Understand and use *args.
- Understand and use **kwargs.
- Identify common function errors and improve the solution.
- Refactor earlier loop programs into reusable functions.

How to Use This Assignment

1. Read and understand the concept before coding.
2. Write a small example before moving to the next concept.
3. Predict what the function will return before running it.
4. Try changing the arguments and observe the result.
5. Refactor repeated code instead of copying and pasting.
6. If you use AI for help, understand and explain the final solution yourself.
7. Keep your final reusable functions in the utility module.

PART A – Understand Functions

1. What is a Function?

Think about: Why put code inside a function instead of writing it every time?

ANSWER: Because it is time waste to write every time when you need that function call from anywhere.to reduce the time complexity and space complexity.

```
def greet():
    print("Hello!")

greet()
greet()----->Hello!

Hello!
```

Explain: What is the function? What is the function call? How many times does it execute?

Answer : function is a block of code to solve the problem instead of writing multiple of time you can write once once run many times.function is nothing but we call the function when we need it.how many time we call that times of time it can run .

2. Function with a Parameter

```
def greet(name):
    print("Hello", name)

greet("Ravi")----->Hello Ravi
greet("Anita")----->Hello Anita
```

Experiment by changing the values passed to name.

Greet("Pavithra")----->Hello Pavithra

3. Return Value

```
def add(a, b):
    return a + b

result = add(10, 20)
print(result)----->30
```

Explain the difference between print and return.

Return ----->give to back to the function.

Print----->it displays the in output screen.

4. Multiple Parameters

```
def calculate_total(price, quantity):
    return price * quantity

print(calculate_total(100, 3))----->300
```

Explain what each parameter represents.

Price ----->100

Quantity----->3

PART B – Parameters vs Arguments

- Parameter: the variable written in the function definition.
- Argument: the actual value passed when calling the function.

```
def introduce(name, age):
    return f"{name} is {age} years old"
```

introduce("Rahul", 21)-----> name and age are parameters

And Rahul and 21 are arguments

Write two examples of your own and label parameters and arguments.

```
def random(a):
```

```
    Print(a)
```

Random(15)-----> a is parameter and 15 is arguments

```
def grade_of_student(grade)
```

```
    return( grade)
```

Print(grade("A"))----->grade parameter and A is arguments

PART C – Default Arguments

```
def greet(name, city="Hyderabad"):
    return f"{name} lives in {city}"
```

print(greet("Ravi"))----->Ravi lives in Hyderabad

print(greet("Anita", "Bangalore"))----->Anita lives in Bangalore

- What happens when city is not provided?
It take default value Hyderabad
- What happens when city is provided?
It take which is provide by us.
- Why are default arguments useful?
Most of the argument is that value but rarely changes so we use the default argument.

Create at least 2 functions using default arguments.

```
def introduce(name,age=21)
```

```
    print(name,"age is ",age)
```

introduce("pavithra")----->pavithra age is 21

introduce("Venky",20)----->Venky age is 20

```
def cal_sum(a,b=10)
```

```
    return a+b
```

```
total=cal_sum(23)
```

Print(total)----->33 where the default b value is 10

PART D – *args

Understand *args as a way to receive a variable number of positional arguments.

```
def add_all(*numbers):
    total = 0
    for number in numbers:
        total += number
    return total

print(add_all(10, 20))----->30
print(add_all(10, 20, 30, 40))----->100
```

Experiment with 0, 1, 2 and many arguments. Explain what *args gives you inside the function.

Because the user want some time to sum 2 numbers some time more then 2 so when you give the *number it take how many number he want from his.

PART E – **kwargs

Understand **kwargs as a way to receive a variable number of keyword arguments.

```
def show_profile(**details):
    for key, value in details.items():
        print(key, ":", value)

show_profile(name="Ravi", age=21, city="Hyderabad")
```

Output:

name:Ravi

age:21

city:Hyderabad

Add/remove profile details and explain what data you receive inside the function.

If we want to add additional details we can pass when we call it

If we want to remove the details we don't pass the details when we call it

We can receive the data in the dictionary type

PART F – Combine the Concepts

```
def create_profile(name, city="Hyderabad", *skills, **details):
    print("Name:", name)
    print("City:", city)
    print("Skills:", skills)
    print("Other Details:", details)
```

```
create_profile("Ravi", "Bangalore", "Python", "React", age=21, college="ABC Engineering College")
```

Output:

Name:Ravi

City:Bangalore

```
Skills: (Python, React)
```

```
Other Details: {age=21, college="ABC Engineering College"}
```

Modify this example and explain what each part receives.

```
create_profile("Pavithra", "Nellore", "Python", "java", age=21, college="PEC")
```

Output:

```
Name: Pavithra
```

```
City: Nellore
```

```
Skills: (Python, java)
```

```
Other Details: {age=21, college="PEC"}
```

Where the pavithra is argument for the name ,Nellore argument for the city ,python and java for skills and PEC for the collage .

PART G – Predict Before You Run

Predict the output first, then run each program.

```
def square(number):  
    return number * number
```

```
x = square(5)  
print(x)----->3125
```

```
def greet(name="Student"):  
    return "Hello " + name
```

```
print(greet())----->Hello Student  
print(greet("Phani"))----->Hello phani
```

```
def total(*numbers):  
    return sum(numbers)
```

```
print(total(1, 2, 3))----->6  
print(total(10, 20, 30, 40))----->100
```

```
def profile(**data):  
    return data
```

```
result = profile(name="Ravi", age=21)  
print(result["name"])----->"Ravi"
```

```
print(result.keys())----->name
```

```
Age
```

```
print(result.values())----->Ravi
```

```
21
```

```
print(result.items())----->name Ravi
```

```
Age 21
```

PART H – Common Function Errors

For every bug: identify the problem, explain why it happens, fix it and test it.

```
Bug 1
def add(a, b)
    return a + b
```

It can show us result .because the return give back to the function it can't display on the screen so we need to call and store in the variable and print that variable.

```
def add(a, b)
    return a + b

total=add(2,4)

print(total)----->6
```

What is missing?

```
Bug 2
def add(a, b):
    print(a + b)

result = add(10, 20)
print(result)-----it can store anything in the result
```

Here it already display the result in the screen ,but when you use the function actually to return will use.

```
def add(a, b):
    return a + b

result = add(10, 20)
print(result)
```

Explain print vs return.

return give back to the function and print display in the screen

```
Bug 3
def greet(name):
    return "Hello " + name

print(greet())
```

Why does this fail? Fix it using a default or supplied argument.

Here when you call the function we can give the arguments or that function have default arguments id okay.

```
def greet(name="Venky"):
    return "Hello " + name

print(greet())----->Hello Venky
```

```
Bug 4
def calculate(price, quantity=1):
```

```

        return price * quantity

print(calculate())

```

Why does this still fail?

We are given default value to only the quantity not the price so, need to pass the argument when it calls

```

def calculate(price, quantity=1):
    return price * quantity

print(calculate(100))----->100

```

```

Bug 5
def add_all(*numbers):
    return numbers + 10

```

What is numbers? Fix the function.

Where numbers is collect the multiple positional arguments.

```

def add_all(*numbers):
    return sum(numbers) + 10

print(add_all(6,4))----->20

```

```

Bug 6
def show(**details):
    print(details)

show("Ravi", 21)

```

Why is this not a correct use of **kwargs? Fix the call.

Ravi and 21 is the positional arguments .but the function parameters are keyword arguments

We can pass arguments like name="pavithra",age=21.

PART I – Refactor Previous Loop Programs

Take your Day 5 loop programs and convert them into reusable functions. Refactor at least:

Multiplication table

```

def multiplication_table(number):
    table=""
    for i in range(11):
        table +=f"{number} x {i} = {i*number}\n"
    return table

```

```
result=multiplication_table(7)
print(result)
```

Sum of numbers

```
def cal_sum(*numbers):
    return sum(numbers)
print(cal_sum(5,9,7,8,90,76))
```

even number checker

```
def even_num(*numbers):
    evennumbers=""
    for number in numbers:
        if number%2 == 0:
            evennumbers +=str(number)+" "
    return evennumbers
print(even_num(56,8,9,90,87,89,85,76,89))
```

Prime number checker

```
def prime_num(number):
    for i in range(2,number):
        if number%i==0:
            return "not a prime number"
    return "prime number"
print(prime_num(10))
```

Palindrome checker

```
def palindrome(word):
    r_word=reversed(word)
    if word==r_word:
        return "palindrome"
```



```
    return "not palindrome"
print(palindrome('pavithra'))
```

Find factors

```
def factors_num(number):
    fact=""
    for i in range(1,number+1):
        if number%i==0:
            fact +=str(i)+" "
    return fact
print(factors_num(86))
```

Pattern printing

```
def pattern_piramid(number):
    for i in range (1,number+1):
        for j in range(number-i):
            print(" ",end="")
        for k in range(2*i-1):
            print("*",end="")
        print()
pattern_piramid(6)
```

Example:

```
# Before
number = 7
for i in range(1, 11):
    print(number * i)
```

Output:-

```
7
14
21
28
35
```

42

49

56

63

70

After

```
def multiplication_table(number):  
    for i in range(1, 11):  
        print(number * i)
```

```
multiplication_table(7)----->7 14 21 28 35 42 49 56 63 70
```

```
multiplication_table(12)----->12 24 36 48 60 72 84 96 108 120
```

Do not merely put old code inside def. Ask: What should the function receive? What should it return? Can another program reuse it?

Yes the this function be reuse for another numbers

It displays the multiplication because we use print

PART J – Function Practice Problems

Problem 1 – Temperature Converter

Create functions for Celsius → Fahrenheit and Fahrenheit → Celsius.

```
def F_C(temp_in_cel):  
    temp_in_fah=(temp_in_cel*(9/5))+32  
    return temp_in_fah  
print(F_C(32))  
def C_F(temp_in_fah):  
    temp_in_cal=(temp_in_fah-32)*(5/9)  
    return temp_in_cal  
print(C_F(43))
```

Output:-

89.6

6.111111111111112

Problem 2 – Even/Odd Function

Create a function that receives a number and returns whether it is even or odd.

```
def even_odd(*numbers):  
    evennumbers=""  
    oddnumber=""  
    for number in numbers:  
        if number%2 == 0:  
            evennumbers +=str(number)+" "  
        else:  
            oddnumber +=str(number)+" "  
    return evennumbers , oddnumber  
print(even_odd(56,8,9,90,87,89,85,76,89))
```

Output:-

('56 8 90 76 ', '9 87 89 85 89 ')

Problem 3 – Prime Function

Create reusable is_prime(number) returning True or False.

```
def prime_num(number):  
    for i in range(2,number):  
        if number%i==0:  
            return "False"  
    return "True"  
print(prime_num(10))
```

Output:-

False

Problem 4 – Palindrome Function

Create reusable is_palindrome(value).

```
def palindrome(word):  
    r_word=reversed(word)  
    if word==r_word:
```

```
        return "palindrome"
    return "not palindrome"
print(palindrome('pavithra'))
```

Output:-

not palindrome

Problem 5 – Grade Function

Create a function that receives marks and returns a grade; handle invalid marks.

```
def calculate_grade(mark=50):
    if mark>=90 and mark<=100:
        return "A"
    elif mark>=75 and mark<=89:
        return "B"
    elif mark>=60 and mark<=74:
        return "C"
    elif mark>=50 and mark<=59:
        return "D"
    else:
        return "Fail"
grade=calculate_grade(87)
```

Output:-

B

Problem 6 – Shopping Total

Create a function receiving price and quantity and returning the total.

```
def shopping_total(price,quantity=1):
    total=price*quantity
    return total
print("total bill:",shopping_total(23,8))
```

Output:-total bill: 184

Problem 7 – Flexible Sum

Create add_all(*numbers) returning the sum of any number of values.

```
def add_all(*numbers):
    total=0
    for number in numbers:
        total +=number
    return total
result=add_all(8,9,90,87,45)
print("the total sum :",result)
```

Output:-

the total sum : 239

Problem 8 – Profile Builder

Create build_profile(**details) returning a profile dictionary.

```
def student_details(name,age=21,*subject,**student_data):
    print(f"Name :{name}")
    print(f"Age :{age}")
    print(f"Subjects :{subject}")
    for key,value in student_data.items():
        print(f"{key} : {value}")
student_details("pavithra" ,22, "java" ,"python", "c",surname="Billupati",city="nellore")
```

Output:-

Name :pavithra

Age :22

Subjects :('java', 'python', 'c')

surname : Billupati

city : nellore

Problem 9 – Greeting Utility

Create a greeting function with a default city and optional name.

```
def greeting(name="pavithra",city="nellore"):
    return f"Hello {name},welcome to {city}"
print(greeting("Venky"))
print(greeting("Somu", "Anamthasagram"))
```

Output:-

Hello Venky,welcome to nellore

Hello Somu,welcome to Anamthasagram

Problem 10 – Statistics Utility

Create functions for minimum, maximum and average of a collection of numbers.

```
def minimum_num(*numbers):  
    min_num=min(numbers)  
    return min_num  
  
def maximum_num(*numbers):  
    max_num=max(numbers)  
    return max_num  
  
def average(*numbers):  
    return sum(numbers)/len(numbers)  
  
print("the minimum number ",minimum_num(3,8,2,89,76,1))  
print("the maximum number ",maximum_num(78,89,23,70,98,90))  
print("the average is",average(4,8,97,76,98,56,23))
```

Output:-

the minimum number 1

the maximum number 98

the average is 51.714285714285715

PART L – Explain Your Code

Why are functions useful?

Without writing again and again we can call the function when it need

What is the difference between a parameter and an argument?

Parameters are when you create the function .the arguments are actually the real values pass by the function

What is the difference between print and return?

Return give back to the function.print displays the output in the screen

Why use a default argument?

A default argument is used when the user does not pass a value for that argument.the function uses the dealt value instead.

What does *args allow you to do?

Is used to pass multiple values to a function .inside the function the values are stored as tuple

What does **kwargs allow you to do?

Is used to pass multiple key and values to a function inside the function the keys and values are stored as a dict.

Why is refactoring useful?

Refactoring is useful because to make the code easier,cleaner and avoiding of rewriting the code.

Submission Checklist

- ☐ Functions explained in my own words.✓
- ☐ Parameters and arguments practiced.✓
- ☐ Return values practiced.✓
- ☐ Default arguments practiced.✓
- ☐ *args practiced.✓
- ☐ **kwargs practiced.✓
- ☐ Debugging completed.✓
- ☐ 10 function problems completed.✓
- ☐ Previous loop programs refactored.✓
- ☐ Utility module contains at least 10 functions.✓
- ☐ Utility module contains default args.✓
- ☐ Utility module contains *args.✓
- ☐ Utility module contains **kwargs.✓
- ☐ Functions tested.✓
- ☐ Both files are ready to submit.✓

Suggested Evaluation

Area	Weight
Concept understanding + explanation	15%
10 Function Problems	25%
Refactoring Day 5 Programs	20%
Utility Module	25%
Testing + Debugging	10%
Code Quality + Reflection	5%

DAY 6 PRINCIPLE

Don't write the same logic again and again. Write it once, make it reusable, and call it whenever you need it.